

Practical No: 1

Name: Agam Mayur Ashok

Roll No: 01

Year & Branch: T.E.(AI&DS)

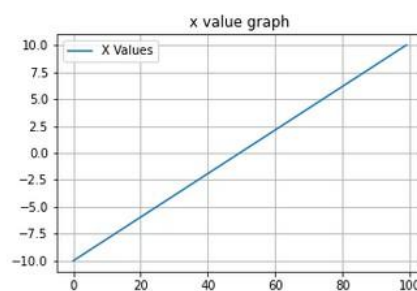
Problem Statement: Write a Python program to plot a few activation functions that are being used in neural networks.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(-10,10,100)
x
plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.plot(x,label='X Values')
plt.title('x value graph')
plt.grid()
plt.legend()
```

```
In [2]: plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.plot(x,label='X Values')
plt.title('x value graph')
plt.grid()
plt.legend()

Out[2]: <matplotlib.legend.Legend at 0x7f1ef27894b0>
```



Sigmoid Activation Function:

```
def sigmoid(x):
    return 1/(1+np.exp(-x))
sigmoid_values=sigmoid(x)

#Plot the activation function
plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.plot(x, sigmoid_values, label='Sigmoid Values')
```

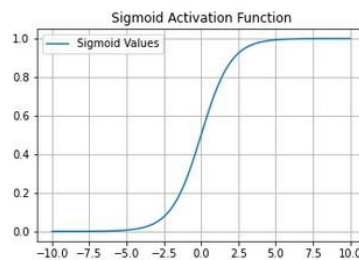
```
plt.title('Sigmoid Activation Function')
plt.grid()
plt.legend()
```

```
In [6]: def sigmoid(x):
        return 1/(1+np.exp(-x))

        sigmoid_values=sigmoid(x)

        #Plot the activation function
        plt.figure(figsize=(12,8))
        plt.subplot(2,2,1)
        plt.plot(x, sigmoid_values, label='Sigmoid Values')
        plt.title('Sigmoid Activation Function')
        plt.grid()
        plt.legend()
```

Out[6]: <matplotlib.legend.Legend at 0x7f1ef26a1510>



Tanh Activation Function:

```
def tanh(x):
    return (np.exp(x) - np.exp(-x))/(np.exp(x) + np.exp(-x))
tanh_values=tanh(x)
```

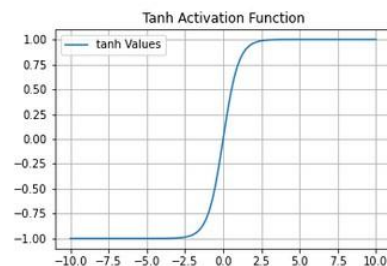
```
#Plot the activation function
plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.plot(x, tanh_values, label='tanh Values')
plt.title('Tanh Activation Function')
plt.grid()
plt.legend()
```

```
In [3]: def tanh(x):
        return (np.exp(x) - np.exp(-x))/(np.exp(x) + np.exp(-x))

        tanh_values=tanh(x)

        #Plot the activation function
        plt.figure(figsize=(12,8))
        plt.subplot(2,2,1)
        plt.plot(x, tanh_values, label='tanh Values')
        plt.title('Tanh Activation Function')
        plt.grid()
        plt.legend()
```

Out[3]: <matplotlib.legend.Legend at 0x7f1ef27d7430>



ReLU Activation Function:

```
def ReLU(x):
```

```
    return np.maximum(0,x)
```

```
ReLU_values = ReLU(x)
```

```
#Plot the activation function
```

```
plt.figure(figsize=(12,8))
```

```
plt.subplot(2,2,1)
```

```
plt.plot(x, ReLU_values, label='ReLU Values')
```

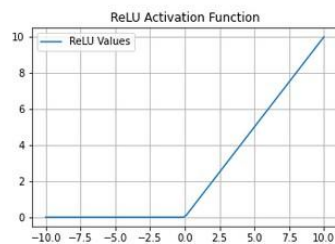
```
plt.title('ReLU Activation Function')
```

```
plt.grid()
```

```
plt.legend()
```

```
In [4]: def ReLU(x):  
        return np.maximum(0,x)  
  
        ReLU_values = ReLU(x)  
  
        #Plot the activation function  
        plt.figure(figsize=(12,8))  
        plt.subplot(2,2,1)  
        plt.plot(x, ReLU_values, label='ReLU Values')  
        plt.title('ReLU Activation Function')  
        plt.grid()  
        plt.legend()
```

```
Out[4]: <matplotlib.legend.Legend at 0x7f1ef25f0eb0>
```



Leaky ReLU Activation Function:

```
def Leaky_ReLU(x, alpha=0.01):
```

```
    return np.where(x>0,x,alpha*x)
```

```
values = Leaky_ReLU(x)
```

```
#Plot the activation function
```

```
plt.figure(figsize=(12,8))
```

```
plt.subplot(2,2,1)
```

```
plt.plot(x, values, label='Leaky ReLU Values')
```

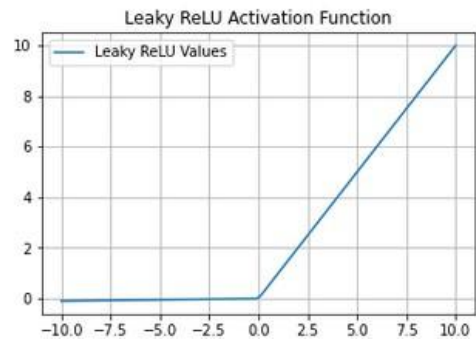
```
plt.title('Leaky ReLU Activation Function')
```

```
plt.grid()
```

```
plt.legend()
```

```
In [5]: def Leaky_ReLU(x, alpha=0.01):  
        return np.where(x>0,x,alpha*x)  
  
        values = Leaky_ReLU(x)  
  
        #Plot the activation function  
        plt.figure(figsize=(12,8))  
        plt.subplot(2,2,1)  
        plt.plot(x, values, label='Leaky ReLU Values')  
        plt.title('Leaky ReLU Activation Function')  
        plt.grid()  
        plt.legend()
```

Out[5]: <matplotlib.legend.Legend at 0x7f1ef266bc70>



Practical No: 02

Name: Agam Mayur Ashok

Roll No: 01

Year & Branch: T.E(AI&DS)

Problem Statement : Generate ANDNOT function using McCulloch-Pitts neural net by a python program.

Source Code:

```
def mc_culloch_pitts_andnot(x1, x2):
    #assign weights
    w1 = 1
    w2 = -1
    #threshold
    theta = 1
    #calculate weighted sum
    S = (x1 * w1) + (x2 * w2)
    #step activation function
    if S >= theta:
        return 1
    else:
        return 0
#Testing AND-NOT Function
print("ANDNOT(0,0) = ", mc_culloch_pitts_andnot(0,0))
print("ANDNOT(0,1) = ", mc_culloch_pitts_andnot(0,1))
print("ANDNOT(1,0) = ", mc_culloch_pitts_andnot(1,0))
print("ANDNOT(1,1) = ", mc_culloch_pitts_andnot(1,1))
```

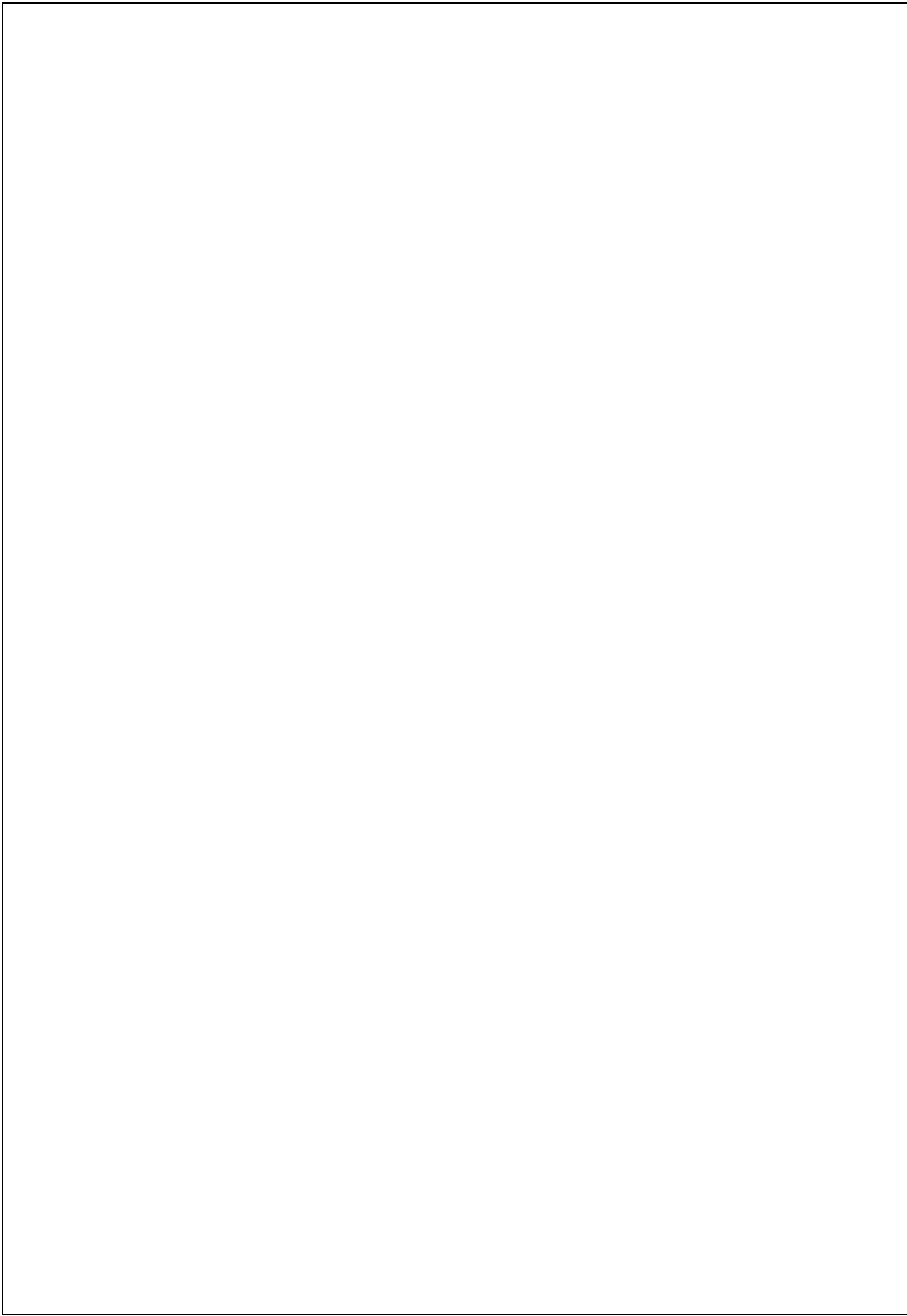
Output:



```
In [2]: def mc_culloch_pitts_andnot(x1, x2):
        #assign weights
        w1 = 1
        w2 = -1
        #threshold
        theta = 1
        #calculate weighted sum
        S = (x1 * w1) + (x2 * w2)
        #step activation function
        if S >= theta:
            return 1
        else:
            return 0
        #Testing AND-NOT Function
        print("ANDNOT(0,0) = ", mc_culloch_pitts_andnot(0,0))
        print("ANDNOT(0,1) = ", mc_culloch_pitts_andnot(0,1))
        print("ANDNOT(1,0) = ", mc_culloch_pitts_andnot(1,0))
        print("ANDNOT(1,1) = ", mc_culloch_pitts_andnot(1,1))

        ANDNOT(0,0) = 0
        ANDNOT(0,1) = 0
        ANDNOT(1,0) = 1
        ANDNOT(1,1) = 0

In [ ]:
```



Practical no: 03

Name: Agam Mayur Ashok

Roll no: 01

Year & Branch: T.E(AI&DS)

Problem Statement:

With a suitable example demonstrate the perceptron learning law with its decision regions using python. Give the output in graphical form.

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt

#Generate random 2D
points X =
np.random.randn(50, 2)

# Label: if  $x_1 + x_2 > 0 \rightarrow +1$  else -1
y = np.array([1 if  $x_1 + x_2 > 0$  else -1 for  $x_1, x_2$  in X])

# Start with zero
weights w =
np.zeros(2)
b = 0
lr = 0.1

# Training for few
iterations for _ in
range(15):
    for i in range(len(X)):
        if  $y[i] * (np.dot(w, X[i]) + b) \leq 0$  : # Wrong
            prediction w += lr *  $y[i] * X[i]$ 
            b += lr *  $y[i]$ 

# Plot points
plt.scatter(X[:,0], X[:,1], c=y, cmap='bwr')

# Plot learned
line x =
np.array([-2, 2])
y_line =  $-(w[0]*x + b) / w[1]$ 
plt.plot(x, y_line)
# Update professional title
plt.title("Perceptron
Classifier")
plt.show()
```

Output:

```
[6]: import numpy as np
import matplotlib.pyplot as plt

#Generate random 2D points
X = np.random.randn(50, 2)

# Label: if x1 + x2 > 0 -> +1 else -1
y = np.array([1 if x1 + x2 > 0 else -1 for x1, x2 in X])

# Start with zero weights
w = np.zeros(2)
b = 0
lr = 0.1

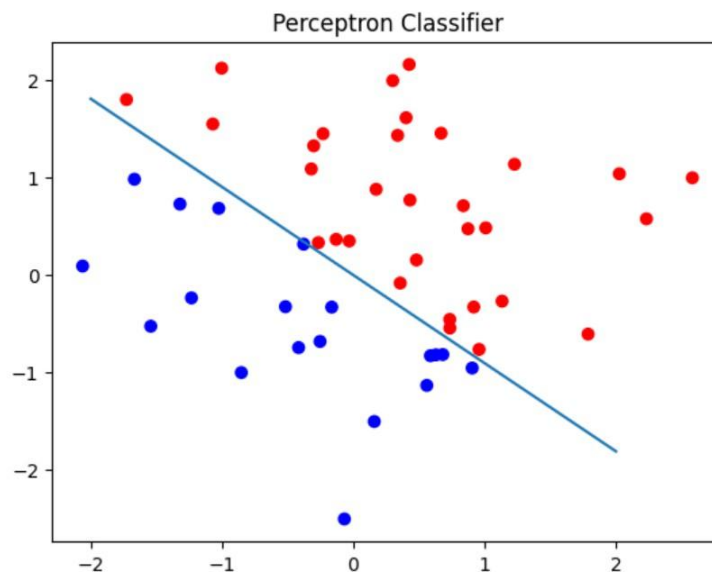
# Training for few iterations
for _ in range(15):
    for i in range(len(X)):
        if y[i] * (np.dot(w, X[i]) + b) <= 0 : # Wrong prediction
            w += lr * y[i] * X[i]
            b += lr * y[i]

# Plot points
plt.scatter(X[:,0], X[:,1], c=y, cmap='bwr')

# Plot Learned line
x = np.array([-2, 2])
y_line = -(w[0]*x + b) / w[1]
plt.plot(x, y_line)

# Update professional title
plt.title("Perceptron Classifier")

plt.show()
```



Practical No :- 04

Name :- Agam Mayur Ashok

Roll No. 01

Class :- TE (AI&DS)

Problem statement :- Write a Python Program using Perceptron Neural Network to recognise even and odd numbers. Given numbers are in ASCII form 0 to 9

Source Code :-

```
import numpy as np

# ASCII - 8 bit binary
def ascii_to_binary(a):
    return np.array([int(i) for i in format(a, '08b')])

# Perceptron training
def train(X, y):
    w = np.zeros(8)
    b = 0

    for epoch in range(100):
        for xi, t in zip(X, y):

            output = 1 if np.dot(w, xi) + b >= 0 else -1

            if output != t:
                w = w + t * xi
                b = b + t

    return w, b

# Digits for training
digits = "0123456789"

# Input data
X = np.array([ascii_to_binary(ord(d)) for d in digits])

# Labels (even = 1, odd = -1)
y = np.array([1 if int(d) % 2 == 0 else -1 for d in digits])

# Training
w, b = train(X, y)

# ----- User Input -----
num = input("Enter a digit (0-9): ")

# Convert input to binary vector
xi = ascii_to_binary(ord(num))
```

```
# Perceptron prediction
out = 1 if np.dot(w, xi) + b >= 0 else -1

if out == 1:
    print("Number is Even")
else:
    print("Number is Odd")
```

OUTPUT :-

```
[14]: import numpy as np
```

```
[15]: # ASCII - 8 bit binary
def ascii_to_binary(a):
    return np.array([int(i) for i in format(a, '08b')])
```

```
[18]: # Perceptron training
def train(X, y):
    w = np.zeros(8)
    b = 0

    for epoch in range(100):
        for xi, t in zip(X, y):
            output = 1 if np.dot(w, xi) + b >= 0 else -1

            if output != t:
                w = w + t * xi
                b = b + t

    return w, b
```

```
[20]: # Digits for training
digits = "0123456789"
```

```
[21]: # Input data
X = np.array([ascii_to_binary(ord(d)) for d in digits])
```

```
[22]: X
```

```
[22]: array([[0, 0, 1, 1, 0, 0, 0, 0],
        [0, 0, 1, 1, 0, 0, 0, 1],
        [0, 0, 1, 1, 0, 0, 1, 0],
        [0, 0, 1, 1, 0, 0, 1, 1],
        [0, 0, 1, 1, 0, 1, 0, 0],
        [0, 0, 1, 1, 0, 1, 0, 1],
        [0, 0, 1, 1, 0, 1, 1, 0],
        [0, 0, 1, 1, 0, 1, 1, 1],
        [0, 0, 1, 1, 1, 0, 0, 0],
        [0, 0, 1, 1, 1, 0, 0, 1]])
```

```
r 1.
```

```
[23]: y = np.array([1 if int(d) % 2 == 0 else -1 for d in digits])
```

```
[24]: w, b = train(X, y)
```

```
[25]: num = input("Enter a digit (0-9): ")

xi = ascii_to_binary(ord(num))

out = 1 if np.dot(w, xi) + b >= 0 else -1

if out == 1:
    print("Number is Even")
else:
    print("Number is Odd")
```

```
Enter a digit (0-9): 5
Number is Odd
```

Practical No :- 05

Name :- Agam Mayur Ashok

Roll No.:- 01

Class :- TE (AI&DS)

Problem statement :- Write a python program to design a Hopfield Network which stores 4 vectors

Source Code :-

```
import numpy as np
patterns = [
    [1,-1,1,-1],
    [-1,1,-1,1],
    [1,1,-1,-1],
    [-1,-1,1,1]
]

size = 4

def train_hopfield(patterns,size):
    weights = np.zeros((size,size))
    for p in patterns:
        p = np.array(p).reshape(size,1)
        weights = weights + np.dot(p,p.T)
        np.fill_diagonal(weights,0)
    return weights

train_hopfield(patterns,size)
```

OUTPUT :-

```
[27]: import numpy as np

[28]: patterns = [
        [1,-1,1,-1],
        [-1,1,-1,1],
        [1,1,-1,-1],
        [-1,-1,1,1]
    ]

[29]: size = 4

[30]: def train_hopfield(patterns,size):
        weights = np.zeros((size,size))
        for p in patterns:
            p = np.array(p).reshape(size,1)
            weights = weights + np.dot(p,p.T)
            np.fill_diagonal(weights,0)
        return weights

[31]: train_hopfield(patterns,size)

[31]: array([[ 0.,  0.,  0., -4.],
             [ 0.,  0., -4.,  0.],
             [ 0., -4.,  0.,  0.],
             [-4.,  0.,  0.,  0.]])
```

Practical No : 6

Name : Agam Mayur Ashok

Roll No : 01

Class : TE(AI&DS)

Problem Statement : Write a python Program for Bidirectional Associative Memory with two pairs of vectors.

Code :

```
import numpy as np
x = np.array([
    [1, -1, 1],
    [-1, 1, -1]
])
y = np.array([
    [1, 1, -1],
    [-1, -1, 1]
])
w = np.dot(x.T,y)

def recall(input_vector, weight_matrix, is_forward=True):
    output = np.sign(np.dot(input_vector, weight_matrix))
    output[output==0]=1
    return output

test_x = np.array([[1, -1, 1]])
recalled_y = recall(test_x, w, is_forward=True)
print("Recalled y for given x:", recalled_y)

test_y = np.array([[1, 1, -1]])
recalled_x = recall(test_y, w.T, is_forward=False)
print("Recalled x for given y:", recalled_x)
```

Output:

```
[1]: import numpy as np
```

```
[2]: x = np.array([
      [1, -1, 1],
      [-1, 1, -1]
    ])
```

```
[4]: y = np.array([
      [1, 1, -1],
      [-1, -1, 1]
    ])
```

```
[7]: w = np.dot(x.T, y)
```

```
[15]: def recall(input_vector, weight_matrix, is_forward=True):
      output = np.sign(np.dot(input_vector, weight_matrix))
      output[output==0]=1
      return output
```

```
[16]: test_x = np.array([[1, -1, 1]])
      recalled_y = recall(test_x, w, is_forward=True)

      print("Recalled y for given x:", recalled_y)
```

Recalled y for given x: [[1 1 -1]]

```
[17]: test_y = np.array([[1, 1, -1]])
      recalled_x = recall(test_y, w.T, is_forward=False)

      print("Recalled x for given y:", recalled_x)
```

Recalled x for given y: [[1 -1 1]]

Practical No :07

Name :Agam Mayur Ashok

Roll No. 01

Class :TE(AI&DS)

Problem Statement:Write a python program to illustrate ART neural network.

Code:

```
import numpy as np
data=np.array([
    [1,1,0,0],
    [1,1,1,0],
    [0,0,1,1],
    [1,0,0,0]
])
rho=0.7
cluster=[]
for sample in data :
    print("\nInput:",sample)

    if len(cluster)==0:
        cluster.append(sample.copy())
        print("Created Cluster 1 done")
        continue
    found = False
    for i in range(len(cluster)):
        w=cluster[i]
        match=np.sum(np.minimum(sample,w))/np.sum(sample)
        print("\n match with cluster: ",i+1,"=",round(match,2))
        if match>=rho:
            print("Added to cluster",i+1)
            cluster[i]=np.minimum(w,sample)
            found=True
            break

    if not found:
        cluster.append(sample.copy())
        print('new Cluster created')
print("\nfinal cluster : ")
for i in range(len(cluster)):
    print('cluster',i+1,':',cluster[i])
```

Output:

```
Input: [1 1 0 0]
Created Cluster 1 done

Input: [1 1 1 0]

  match with cluster: 1 = 0.67
new Cluster created

Input: [0 0 1 1]

  match with cluster: 1 = 0.0

  match with cluster: 2 = 0.5
new Cluster created

Input: [1 0 0 0]

  match with cluster: 1 = 1.0
Added to cluster 1

final cluster :
cluster 1 : [1 0 0 0]
cluster 2 : [1 1 1 0]
cluster 3 : [0 0 1 1]
```

Practical No. : 08

Name : Agam Mayur Ashok

Roll No. : 01

Class : TE(AI&DS)

Problem Statement : Implement Artificial Neural Network training process in Python by using Forward Propagation, Back Propagation

Code :

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

np.random.seed(0)
W1 = np.random.uniform(-1, 1, (2, 2))
W2 = np.random.uniform(-1, 1, (2, 1))
b1 = np.zeros((1, 2))
b2 = np.zeros((1, 1))

lr = 0.5

def forward_propagation(X):
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)
    final_input = np.dot(hidden_output, W2) + b2
    final_output = sigmoid(final_input)
    return hidden_input, hidden_output, final_input, final_output

def backward_propagation(X, y, hidden_output, final_output):
    global W1, W2, b1, b2

    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

    hidden_error = d_output.dot(W2.T)
    d_hidden = hidden_error * sigmoid_derivative(hidden_output)

    W2 += lr * hidden_output.T.dot(d_output)
    W1 += lr * X.T.dot(d_hidden)
    b2 += lr * np.sum(d_output, axis=0, keepdims=True)
    b1 += lr * np.sum(d_hidden, axis=0, keepdims=True)

for epoch in range(10000):
```

```
hidden_input, hidden_output, final_input, final_output = forward_propagation(X)
backward_propagation(X, y, hidden_output, final_output)

if epoch % 1000 == 0:
    loss = np.mean(np.square(y - final_output))
    print(f'Epoch {epoch}: Loss = {loss:.5f}')

_, _, _, final_output = forward_propagation(X)
print("\nFinal outputs after training:")
print(final_output)
```

Output :

```
Epoch 0: Loss = 0.25042
Epoch 1000: Loss = 0.24337
Epoch 2000: Loss = 0.00788
Epoch 3000: Loss = 0.00222
Epoch 4000: Loss = 0.00124
Epoch 5000: Loss = 0.00085
Epoch 6000: Loss = 0.00065
Epoch 7000: Loss = 0.00052
Epoch 8000: Loss = 0.00043
Epoch 9000: Loss = 0.00037
```

Final outputs after training:

```
[[0.01984931]
 [0.98285981]
 [0.9828567 ]
 [0.01775138]]
```

Practical no. : 09

Name : Agam Mayur Ashok

Roll No. : 15

Class : TE(AI&DS)

Problem Statement : Write a python program in python program for creating a Back Propagation Feed-forward neural network

Code :

```
import numpy as np

X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0],
              [1],
              [1],
              [1]])

input_neurons = 2
hidden_neurons = 2
output_neurons = 1

np.random.seed(1)
W1 = np.random.randn(input_neurons, hidden_neurons)
W2 = np.random.randn(hidden_neurons, output_neurons)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

epochs = 10000
lr = 0.1

for i in range(epochs):
    hidden_input = np.dot(X, W1)
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2)
    final_output = sigmoid(final_input)

    error = y - final_output

    d_output = error * sigmoid_derivative(final_output)
    error_hidden = d_output.dot(W2.T)
    d_hidden = error_hidden * sigmoid_derivative(hidden_output)
```

```
W2 += hidden_output.T.dot(d_output) * lr
W1 += X.T.dot(d_hidden) * lr
```

```
print(final_output)
```

Output :

```
[[0.06156428]
 [0.96693118]
 [0.9678494 ]
 [0.98080152]]
```

Practical No:10

Name: Agam Mayur Ashok

Roll no: 01

Class: TE(AI&DS)

Problem Statement: Write a python program in python program for creating a Back Propagation Feed-forward neural network

Code:

```
import numpy as np
x=np.array([[0,0],[0,1],[1,0],[1,1]])
y=np.array([[0],[0],[0],[1]])
input_neu =2
Hidden_neu =2
out_neu =1
np.random.seed(1)
w1=np.random.randn(input_neu,Hidden_neu)
w2 =np.random.randn(Hidden_neu,out_neu)

def sigmoid(x):
    return 1/(1+np.exp(-x))

def sigmoid_derivative(x):
    return x*(1-x)

epoch=10000
lr=0.1
for i in range(epoch):
    Hidden_input =np.dot(x,w1)
    Hidden_output = sigmoid(Hidden_input)

    final_input=np.dot(Hidden_output,w2)
    final_output=sigmoid(final_input)

    error = y-final_output
    d_output = error*sigmoid_derivative(final_output)
    Hidden_error=d_output.dot(w2.T)
    d_Hidden=Hidden_error*sigmoid_derivative(Hidden_output)

    w2+=Hidden_output.T.dot(d_output)*lr
    w1+=x.T.dot(d_Hidden)*lr

print("The final output is :\n",final_output)
```

Output:

The final output is :

[[0.]

[0.]

[0.]

[1.]]

Practical NO :11

Name: Agam Mayur Ashok

Class : TE(AI&DS)

Roll : 01

Problem Statement : TensorFlow/Pytorch implementation of CNN

Code:

```
import tensorflow as tf from tensorflow.keras import
datasets, layers, models import matplotlib.pyplot as
plt
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
x_train=x_train/255.0 x_test=x_test/255.0 model =
models.Sequential()
#1st
model = models.Sequential() model.add(layers.Input(shape=(32,32,3)))
model.add(layers.Conv2D(32,(3,3), padding='same', activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
#2nd model.add(layers.Conv2D(64,(3,3),
padding='same',activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
#3rd
model.add(layers.Conv2D(128,(3,3), padding='same',activation='relu'))
model.add(layers.MaxPooling2D((2,2))) model.add(layers.Flatten())
model.add(layers.Dense(128,activation='relu'))
model.add(layers.Dense(128,activation='relu'))
model.add(layers.Dense(10,activation='softmax'))
model.summary()
```

output:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 128)	262,272
dense_1 (Dense)	(None, 128)	16,512
dense_2 (Dense)	(None, 128)	16,512

Total params: 388,544 (1.48 MB)

Trainable params: 388,544 (1.48 MB)

Non-trainable params: 0 (0.00 B)

```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    x_train,y_train, epochs=10,
    validation_data=(x_test,y_test)
)

```

```

1563/1563 ————— 46s 28ms/step - accuracy: 0.4584 - loss: 1.4945 - val_accuracy: 0.6097 - val_loss: 1.0953
Epoch 2/10
1563/1563 ————— 51s 32ms/step - accuracy: 0.6397 - loss: 1.0241 - val_accuracy: 0.6788 - val_loss: 0.9147
Epoch 3/10
1563/1563 ————— 42s 27ms/step - accuracy: 0.7074 - loss: 0.8356 - val_accuracy: 0.6973 - val_loss: 0.8664
Epoch 4/10
1563/1563 ————— 42s 27ms/step - accuracy: 0.7506 - loss: 0.7150 - val_accuracy: 0.7212 - val_loss: 0.8315
Epoch 5/10
1563/1563 ————— 43s 28ms/step - accuracy: 0.7819 - loss: 0.6267 - val_accuracy: 0.7269 - val_loss: 0.8304
Epoch 6/10
1563/1563 ————— 45s 29ms/step - accuracy: 0.8066 - loss: 0.5462 - val_accuracy: 0.7437 - val_loss: 0.7887
Epoch 7/10
1563/1563 ————— 49s 32ms/step - accuracy: 0.8319 - loss: 0.4804 - val_accuracy: 0.7355 - val_loss: 0.8150
Epoch 8/10
1563/1563 ————— 78s 50ms/step - accuracy: 0.8492 - loss: 0.4266 - val_accuracy: 0.7339 - val_loss: 0.8438
Epoch 9/10
1563/1563 ————— 83s 53ms/step - accuracy: 0.8714 - loss: 0.3680 - val_accuracy: 0.7241 - val_loss: 0.9339
Epoch 10/10
1563/1563 ————— 83s 53ms/step - accuracy: 0.8841 - loss: 0.3235 - val_accuracy: 0.7458 - val_loss: 0.8655

```

```

test_loss,test_acc=model.evaluate(x_test,y_test)
print("Test Accuracy:",test_acc) prediction =
model.predict(x_test[0].reshape(-1,28,28,1))
predicted_class = tf.argmax(prediction[0]).numpy()
actual_class = y_test[0] print("Predicted class:",
predicted_class)
print("Actual class:", actual_class)

```

```

# display plt.imshow(x_test[0],
cmap='gray') plt.title(f'Predicted:
{predicted_class}') plt.axis('off')
plt.show()

```

output:



Practical NO :12

Name: Agam Mayur Ashok

Class : TE(AI&DS)

Roll : 01

Problem Statement : MNIST Handwritten Character Detection using PyTorch, Keras and Tensorflow

Code:

```
import tensorflow as tf from tensorflow.keras import
datasets, layers, models import matplotlib.pyplot as
plt
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
x_train=x_train/255.0 x_test=x_test/255.0 model =
models.Sequential()
#1st
model = models.Sequential()
model.add(layers.Input(shape=(28,28,1)))
model.add(layers.Conv2D(32,(3,3), padding='same',
activation='relu')) model.add(layers.MaxPooling2D((2,2)))
#2nd
model.add(layers.Conv2D(64,(3,3),
padding='same',activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
#3rd model.add(layers.Conv2D(128,(3,3),
padding='same',activation='relu'))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Flatten())
model.add(layers.Dense(128,activation='relu'))
model.add(layers.Dense(128,activation='relu'))
model.add(layers.Dense(10,activation='softmax'))
model.summary() output:
```

```
model.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_6 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_6 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_7 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 7, 7, 64)	0
conv2d_8 (Conv2D)	(None, 7, 7, 128)	73,856
max_pooling2d_8 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten_2 (Flatten)	(None, 1152)	0
dense_6 (Dense)	(None, 128)	147,584
dense_7 (Dense)	(None, 128)	16,512
dense_8 (Dense)	(None, 10)	1,290

Total params: 258,058 (1008.04 KB)

Trainable params: 258,058 (1008.04 KB)

Non-trainable params: 0 (0.00 B)


```

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
)
history = model.fit(
    x_train,y_train,
    epochs=10,
    validation_data=(x_test,y_test)
)

```

```

Epoch 1/10
W0000 00:00:1776763973.294932 5027 cpu_allocator_impl.cc:82] Allocation of 188160000 exceeds 10% of free system memory.
1875/1875 ————— 86s 43ms/step - accuracy: 0.9571 - loss: 0.1375 - val_accuracy: 0.9859 - val_loss: 0.0400
Epoch 2/10
1875/1875 ————— 82s 44ms/step - accuracy: 0.9863 - loss: 0.0443 - val_accuracy: 0.9852 - val_loss: 0.0482
Epoch 3/10
1875/1875 ————— 83s 44ms/step - accuracy: 0.9900 - loss: 0.0323 - val_accuracy: 0.9879 - val_loss: 0.0417
Epoch 4/10
1875/1875 ————— 36s 19ms/step - accuracy: 0.9920 - loss: 0.0267 - val_accuracy: 0.9916 - val_loss: 0.0272
Epoch 5/10
1875/1875 ————— 37s 20ms/step - accuracy: 0.9942 - loss: 0.0189 - val_accuracy: 0.9862 - val_loss: 0.0450
Epoch 6/10
1875/1875 ————— 36s 19ms/step - accuracy: 0.9949 - loss: 0.0175 - val_accuracy: 0.9919 - val_loss: 0.0258
Epoch 7/10
1875/1875 ————— 36s 19ms/step - accuracy: 0.9954 - loss: 0.0149 - val_accuracy: 0.9926 - val_loss: 0.0246
Epoch 8/10
1875/1875 ————— 36s 19ms/step - accuracy: 0.9958 - loss: 0.0138 - val_accuracy: 0.9929 - val_loss: 0.0264
Epoch 9/10
1875/1875 ————— 35s 19ms/step - accuracy: 0.9967 - loss: 0.0107 - val_accuracy: 0.9941 - val_loss: 0.0225
Epoch 10/10
1875/1875 ————— 39s 21ms/step - accuracy: 0.9968 - loss: 0.0102 - val_accuracy: 0.9924 - val_loss: 0.0318

```

```

test_loss,test_acc=model.evaluate(x_test,y_test)
print("Test Accuracy:",test_acc)
prediction = model.predict(x_test[0].reshape(-1,28,28,1))
predicted_class = tf.argmax(prediction[0]).numpy()
actual_class = y_test[0]
print("Predicted class:",
predicted_class)
print("Actual class:", actual_class)

```

```

# display
plt.imshow(x_test[0],
cmap='gray')
plt.title(f'Predicted:
{predicted_class}')
plt.axis('off')
plt.show()

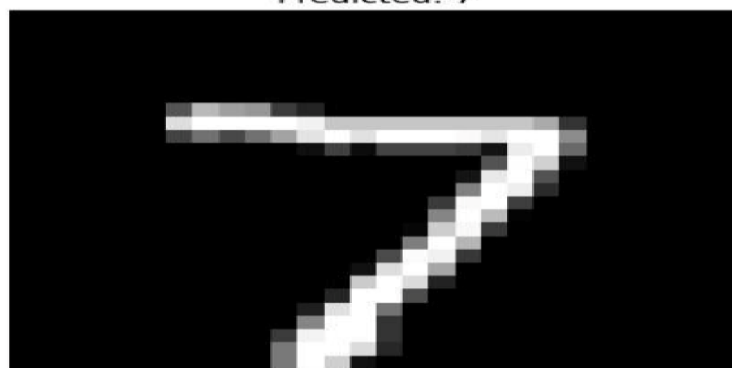
```

output:

```

1/1 ————— 0s 36ms/step
Predicted class: 7
Actual class: 7

```



Practical No. :13

Name : Agam Mayur Ashok

Class : TE[AI&DS]

Roll No. : 01

Problem Statement : MNIST Handwritten Character Detection using PyTorch, Keras and TensorFlow

Code :

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

transform = transforms.ToTensor()
train_loader = DataLoader(datasets.MNIST('./data', train=True, download=True,
transform=transform), batch_size=32, shuffle=True)
test_loader = DataLoader(datasets.MNIST('./data', train=False, download=True,
transform=transform), batch_size=32)

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, 3) # 8 filters, 3x3 kernel
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(8 * 13 * 13, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x))) # Conv + ReLU + Pool
        x = x.view(-1, 8 * 13 * 13) # Flatten
        return self.fc1(x) # Fully connected

model = SimpleCNN()
loss_fn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
for epoch in range(3):
    for images, labels in train_loader:
        preds = model(images)
        loss = loss_fn(preds, labels)
```

```

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        print(f'Epoch {epoch+1} done')

correct, total = 0, 0
with torch.no_grad():
    for images, labels in test_loader:
        preds = model(images)
        correct += (preds.argmax(1) == labels).sum().item()
        total += labels.size(0)
print(f'Accuracy: {100 * correct / total:.2f}%')

# Tensorflow (Keras) code
# Step 1: Import Libraries
import tensorflow as tf
from tensorflow.keras import layers, models

# Step 2: Load Dataset (MNIST)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Step 3: Normalize Data
x_train, x_test = x_train / 255.0, x_test / 255.0

# Step 4: Define Neural Network Model
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Step 5: Compile the Model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Step 6: Train the Model
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
# Step 7: Evaluate the Model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f'Test Accuracy: {test_acc}')

```

Output :

```
100%|██████████| 9.91M/9.91M [00:02<00:00, 4.44MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 132kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.24MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 11.2MB/s]
Epoch 1 done
Epoch 2 done
Epoch 3 done
Accuracy: 97.46%
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-
datasets/mnist.npz
11490434/11490434 ————— 2s
0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(**kwargs)
Epoch 1/5
1875/1875 ————— 9s 4ms/step -
accuracy: 0.9248 - loss: 0.2605 - val_accuracy: 0.9565 - val_loss: 0.1484
Epoch 2/5
1875/1875 ————— 5s 3ms/step -
accuracy: 0.9660 - loss: 0.1162 - val_accuracy: 0.9710 - val_loss: 0.0969
Epoch 3/5
1875/1875 ————— 5s 3ms/step -
accuracy: 0.9763 - loss: 0.0799 - val_accuracy: 0.9733 - val_loss: 0.0845
Epoch 4/5
1875/1875 ————— 10s 3ms/step -
accuracy: 0.9814 - loss: 0.0611 - val_accuracy: 0.9728 - val_loss: 0.0880
Epoch 5/5
1875/1875 ————— 6s 3ms/step -
accuracy: 0.9851 - loss: 0.0484 - val_accuracy: 0.9784 - val_loss: 0.0726
313/313 - 1s - 2ms/step - accuracy: 0.9784 - loss: 0.0726
Test Accuracy: 0.9783999919891357
```